

Úloha A (Error Rate Counters)

Zadanie GNU Radio – Implementácia meračov chybovosti (Variant B, Úloha A)

Úvod a analýza problému

Mojou úlohou v tejto časti bolo vytvoriť vlastný merací blok v Pythone, ktorý by dokázal v reálnom čase sledovať kvalitu prenosu v schéme Variant_B_Error_Rate.grc. Cieľom nebolo len slepo niečo prepojiť, ale nahradiť pôvodne neprehľadné zapojenie jediným efektívnym blokom, ktorý mi naraz vypočíta bitovú (BER), symbolovú (SER) aj rámcovú (FER) chybovosť.

Pri práci som narazil na zásadný problém: pôvodná schéma v GRC bola zapojená tak, že sa program pri pokuse o výpočet neustále zasekával (tzv. deadlock). Analýzou som zistil, že to bolo spôsobené asymetriou dátových tokov – na jeden vstup prichádzali bity a na druhý bajty, čo zahlcovalo vnútornú pamäť GNU Radia.

Riešenie a implementácia

Aby som systém sfunkčnil, musel som urobiť dve kľúčové zmeny:

- Úprava zapojenia: Zistil som, že do môjho bloku musím priviesť „referenčný signál“. Použil som teda blok Virtual Source (napojený na TX Data), ktorý som priviedol do horného vstupu môjho bloku. Tým som zabezpečil, že môj kód má s čím porovnávať prichádzajúce zašumené dáta.
- Prechod na asynchrónny kód: Pôvodný sync_block som v Pythone prerobil na basic_block. To mi umožnilo v kóde explicitne povedať programu, že má „skonsumovať“ 8 bitov z jedného vstupu na každý 1 bajt z druhého vstupu. Týmto sa záseky úplne odstránili a simulácia začala bežať plynule.

Logika výpočtov v bloku

V samotnom Python kóde som implementoval tri úrovne kontroly:

- **BER** (Bit Error Rate): Využívam funkciu `np.unpackbits`, aby som porovnal dáta bit po bite. Je to najpresnejší ukazovateľ kvality kanála.
- **SER** (Symbol Error Rate): Keďže simulácia používa 16-QAM moduláciu, viem, že jeden bajt sa skladá z dvoch 4-bitových symbolov. Kód teda rozoberá bajty na „nibble“ (polbajty) a kontroluje integritu týchto symbolov.
- **FER** (Frame Error Rate): Tu kontrolujem celé rámce podľa nastavenej veľkosti (`frame_size = 100`). Ak sa v rámci nájde hoci len jeden chybný bit, celý rámec považujem za stratený.

Výsledky posielam do GUI v logaritmickej mierke ($\log_{10}$). Ak je prenos bezchybný, nastavil som výstup na pevnú hodnotu -10.0, aby budíky v grafe „nepadali“ na nulu, čo by spôsobovalo chyby pri vykresľovaní.

Overenie a zhodnotenie

Správnosť svojho riešenia som si potvrdil pomocou preddefinovaného testovacieho scenára Test Case 1. Po jeho zapnutí budíky okamžite ukázali hodnoty:

- BER: ≈ -2.9
- SER: ≈ -2.3
- FER: 0.0 (čo pri logaritme znamená 100% chybovosť rámcov)

Tieto čísla presne sedia s referenčnými hodnotami zo zadania. Pri prepnutí na bežnú prevádzku a vypnutí šumu hodnoty okamžite klesli na -10.0. Týmto som overil, že môj blok meria presne a schéma je stabilná aj pri dlhodobom behu.

Python kód:

```
import numpy as np
from gnuradio import gr

class blk(gr.basic_block):
    def __init__(self, frame_size=100, bits_per_symbol=4):
        gr.basic_block.__init__(
            self,
            name='Error Rate Counter',
            in_sig=[np.uint8, np.uint8], # in0: Referencia, in1: Prijaté dáta
            out_sig=[np.float32, np.float32, np.float32] # SER, BER, FER
        )
        self.frame_size = frame_size
        self.bits_per_symbol = bits_per_symbol
        self.total_bits = 0
        self.bit_errors = 0
        self.total_symbols = 0
        self.symbol_errors = 0
        self.total_frames = 0
        self.frame_errors = 0

    def general_work(self, input_items, output_items):
        in_ref = input_items[0]
        in_rec = input_items[1]

        # Zistíme, koľko dát môžeme naraz spracovať
        n_input = min(len(in_ref), len(in_rec))
        if n_input < self.frame_size:
            return 0 # Čakáme na dosť dát pre aspoň jeden rámeček
```

```
n_frames = n_input // self.frame_size
data_len = n_frames * self.frame_size
```

```
ref = in_ref[:data_len]
rec = in_rec[:data_len]
```

```
# 1. BER (Bit Error Rate)
```

```
ref_bits = np.unpackbits(ref)
rec_bits = np.unpackbits(rec)
self.bit_errors += np.sum(ref_bits != rec_bits)
self.total_bits += len(ref_bits)
```

```
# 2. SER (Symbol Error Rate) - Pre 16-QAM (4 bity na symbol = 2 symboly v bajte)
```

```
# Porovnáme horné 4 bity a dolné 4 bity každého bajtu
```

```
ref_h, ref_l = (ref >> 4) & 0x0F, ref & 0x0F
rec_h, rec_l = (rec >> 4) & 0x0F, rec & 0x0F
self.symbol_errors += np.sum(ref_h != rec_h) + np.sum(ref_l != rec_l)
self.total_symbols += len(ref) * 2
```

```
# 3. FER (Frame Error Rate)
```

```
ref_f = ref.reshape(-1, self.frame_size)
rec_f = rec.reshape(-1, self.frame_size)
self.frame_errors += np.sum(np.any(ref_f != rec_f, axis=1))
self.total_frames += n_frames
```

```
# Výpočet logaritmov pre budíky (Number Sink)
```

```
# Ak je chybovosť nulová, pošleme -10.0
```

```
ser = self.symbol_errors / self.total_symbols if self.total_symbols > 0 else 1e-10
```

```
ber = self.bit_errors / self.total_bits if self.total_bits > 0 else 1e-10  
fer = self.frame_errors / self.total_frames if self.total_frames > 0 else 1e-10
```

```
output_items[0][:n_frames] = np.log10(ser) if ser > 0 else -10.0  
output_items[1][:n_frames] = np.log10(ber) if ber > 0 else -10.0  
output_items[2][:n_frames] = np.log10(fer) if fer > 0 else -10.0
```

```
self.consume(0, data_len)  
self.consume(1, data_len)  
return n_frames
```

Obrázok výstupu:

Variant B Error Rate

Test Cases 16 QAM:

☒ Test 1: BER_log = -2.9 ; SER_log = -2.3 ; FER_log = 0

☐ Test 2: BER_log = -3.94 ; SER_log = -3.34 ; FER_log = -1

Reset Button

SER
-2.903090

BER
-2.301030

FER
0.000000

Variant B Error Rate

Test Cases 16 QAM:

☐ Test 1: BER_log = -2.9 ; SER_log = -2.3 ; FER_log = 0

☒ Test 2: BER_log = -3.94 ; SER_log = -3.34 ; FER_log = -1

Reset Button

SER
-3.046073

BER
-2.444013

FER
-0.142983

Variant B Error Rate

Test Cases 16 QAM:

☐ Test 1: BER_log = -2.9 ; SER_log = -2.3 ; FER_log = 0

☒ Test 2: BER_log = -3.94 ; SER_log = -3.34 ; FER_log = -1

Reset Button

SER
-3.318384

BER
-2.716324

FER
-0.415294